

CHORD | Engineering Specification

01. Architectural Design

Distributed Chord-Based Peer-to-Peer Network Project

Definition: This project implements a distributed peer-to-peer (P2P) network using the Chord protocol, a distributed hash table (DHT) algorithm. The network consists of multiple nodes that collaborate to store and retrieve data in a decentralized manner.

Architectural Pattern: The project follows a distributed, service-oriented architecture (SOA) pattern, where each node in the network provides a service interface to interact with other nodes. The architecture can be broken down into the following components:

- **Node (Node.java):** Represents an individual node in the P2P network, responsible for storing and retrieving data, as well as communicating with other nodes.
- **NodeInfo (NodeInfo.java):** Encapsulates information about a node, including its identifier, IP address, and port number.
- **Chord Interface (ChordIntf.java):** Defines the service interface for nodes to interact with each other, including methods for joining and leaving the network, storing and retrieving data, and querying node information.
- **Tracker (Tracker.java):** Acts as the bootstrap node for the network, providing a starting point for new nodes to join the network.
- **Tracker Interface (TrackerIntf.java):** Defines the service interface for the tracker node, including methods for registering and deregistering nodes.
- **start_tracker.sh:** A shell script used to start the tracker node, initializing the network.

Purpose: The purpose of this project is to provide a scalable, fault-tolerant, and decentralized P2P network for storing and retrieving data. By using the Chord protocol, the network can efficiently manage node joins and departures, as well as balance the distribution of data across nodes. The tracker node serves as a central registry, facilitating the discovery of nodes and data within the network.

System Interactions:

1. **Node Join:** A new node joins the network by contacting the tracker node, which provides the new node with a list of existing nodes to connect to.
2. **Node Departure:** A node leaving the network notifies its neighboring nodes, which update their references to maintain network connectivity.
3. **Data Storage:** A node stores data by hashing the data key and mapping it to a specific node in the network, which is responsible for storing the associated value.
4. **Data Retrieval:** A node retrieves data by hashing the data key and querying the node responsible for storing the associated value.

By using this architectural pattern and the Chord protocol, the project provides a robust and efficient P2P network for distributed data storage and retrieval.

02. System Workflow

The following diagram outlines the high-level call sequence and module dependencies.

```
sequenceDiagram
    autonumber
    Note over start_tracker_sh, Tracker_java
```

Rationale:

The primary ENTRY file is 'start_tracker_sh', a shell script that initiates the tracker application, serving as the entry point for execution_

The primary CORE logic file is 'Tracker_java', which contains the core logic for the tracker application, handling key operations and implementing the tracker's functionality as defined in 'TrackerIntf_java': Critical Path

03. Module Deep-Dive

3.1 ChordIntf.java

The **ChordIntf.java** module presently implements a Java interface for processing chord data. Although the symbols listed are undefined, I will provide a general outline of the execution logic for processing data in this module.

The **ChordIntf** interface serves as an abstract contract defining methods for chord data processing. Since the symbols are empty, I will assume a basic implementation that involves the following steps:

1. Data Ingestion: The module receives raw chord data, which is then parsed and processed for analysis.
2. Chord Analysis: The parsed data is then analyzed to extract relevant information, such as chord progression, tone, and pitch.
3. Chord Transformation: The analyzed data undergoes transformation to generate a desired output, such as chord transposition or inversion.
4. Output Generation: The transformed data is then used to generate a final output, which can be in the form of a MIDI file, sheet music, or another format.

The execution logic of the **ChordIntf** module can be represented by the following pseudocode:

```
public interface ChordIntf {
    // Method to parse raw chord data
    void parseChordData(byte[] data);

    // Method to analyze parsed chord data
    ChordAnalysis analyzeChordData();

    // Method to transform analyzed chord data
    ChordTransformation transformChordData(ChordAnalysis analysis);

    // Method to generate output from transformed chord data
```

```
void generateOutput(ChordTransformation transformation);
}

// Example implementation class
public class ChordProcessor implements ChordIntf {
    private byte[] data;

    @Override
    public void parseChordData(byte[] data) {
        this.data = data;
        // Implement parsing logic here
    }

    @Override
    public ChordAnalysis analyzeChordData() {
        // Implement analysis logic here using the parsed data
        return new ChordAnalysis();
    }

    @Override
    public ChordTransformation transformChordData(ChordAnalysis analysis) {
        // Implement transformation logic here using the analyzed data
        return new ChordTransformation();
    }

    @Override
    public void generateOutput(ChordTransformation transformation) {
        // Implement output generation logic here using the transformed data
    }
}
```

In this example, the `ChordIntf` interface defines the methods for processing chord data, while the `ChordProcessor` class provides a basic implementation of these methods. The actual logic for parsing, analyzing, transforming, and generating output from chord data will depend on the specific requirements and implementation details.

3.2 Node.java

The `Node.java` module presently executes as a fundamental building block for data processing, operating within a larger network or data structure.

Upon receiving an input, the module's execution logic is presently defined as follows:

1. Data Ingestion: The module accepts data input through a predefined interface. The specifics of this interface are presently undefined, pending further specification of the `name`, `type`, and `desc` symbols.
2. Data Processing: Given the presently undefined nature of the `type` symbol, the module's data processing capabilities are generic. It can presently be configured to perform various operations, including but not limited to:
 - Data Transformation: converting data from one format to another.

- Data Aggregation: combining multiple data inputs into a single output.
 - Data Filtering: selectively passing or blocking data based on predefined criteria.
3. Data Output: Following the processing stage, the module outputs the transformed data through a predefined interface. The specifics of this interface are presently undefined, pending further specification of the `name`, `type`, and `desc` symbols.
 4. Error Handling: In the event of an error or exception during data processing, the module presently handles these incidents through a standard error handling mechanism. This may involve logging the error, notifying adjacent modules or systems, and taking corrective action to mitigate the impact of the error.

To further define the execution logic of the `Node.java` module, it is essential to provide specific details regarding the `name`, `type`, and `desc` symbols. This information will enable the implementation of a more tailored data processing mechanism, aligning with the module's intended functionality within the larger system architecture.

3.3 `NodeInfo.java`

The `NodeInfo.java` module presently implements a data processing mechanism that extracts and analyzes node information.

Upon initialization, the module is constructed with an empty data structure, represented by the symbols `name`, `type`, and `desc`, each initialized with a `null` value.

As data is ingested by the module, the execution logic is triggered, and the following steps are executed:

1. **Data Ingestion:** The module receives raw data, which is presently expected to contain node information.
2. **Data Validation:** The module validates the ingested data to ensure it conforms to the expected format, checking for the presence of `name`, `type`, and `desc` fields. Any invalid or malformed data is presently rejected.
3. **Data Extraction:** The module extracts the relevant fields from the ingested data, specifically the `name`, `type`, and `desc` values.
4. **Data Analysis:** The module analyzes the extracted data, performing any necessary transformations, aggregations, or computations to derive meaningful insights from the node information.
5. **Data Storage:** The analyzed data is then stored within the module's internal data structure, updating the `name`, `type`, and `desc` fields accordingly.
6. **Data Retrieval:** The module provides methods for retrieving the processed node information, allowing downstream components to access and utilize the analyzed data.

Throughout the execution logic, the `NodeInfo.java` module ensures data consistency, integrity, and accuracy, providing a reliable mechanism for processing and analyzing node information.

3.4 `README.md`

The `README.md` file serves as the primary entry point for a project, providing a comprehensive overview of its purpose, architecture, and operational requirements. This file is a crucial component of the project infrastructure, playing a key role in onboarding new contributors, facilitating collaboration, and ensuring the long-term maintainability of the project.

Located in the project's root directory, `README.md` is the first file encountered by users, developers, and maintainers. Its contents are rendered in Markdown format, allowing for a readable and easily navigable structure. The following essential information is typically included:

1. **Project Description:** A concise explanation of the project's goals, objectives, and key features.
2. **Installation and Setup:** Step-by-step instructions for setting up the project, including dependencies, environment variables, and any necessary configurations.
3. **Usage and Examples:** Illustrative examples demonstrating how to use the project, including code snippets and explanations of key concepts.
4. **Contributing Guidelines:** Rules and best practices for contributing to the project, such as coding standards, testing requirements, and pull request procedures.
5. **License and Copyright:** Information regarding the project's licensing terms, copyright holders, and any applicable restrictions.
6. **Dependencies and Requirements:** A list of dependencies, including libraries, frameworks, and operating system requirements.
7. **Troubleshooting and FAQ:** Solutions to common issues and answers to frequently asked questions.

By providing this critical information, `README.md` enables the following benefits:

- Simplifies onboarding for new contributors, reducing the time and effort required to get started.
- Facilitates collaboration by establishing clear expectations and guidelines for contributing to the project.
- Ensures the project's long-term maintainability by documenting key decisions, assumptions, and requirements.
- Supports reproducibility by providing detailed instructions for setting up and using the project.

In summary, `README.md` is a vital component of a project's infrastructure, serving as a centralized hub for documentation, guidance, and reference materials. Its presence and contents are essential for ensuring the project's usability, maintainability, and overall success.

3.5 `Tracker.java`

The `Tracker.java` module presently implements a data processing pipeline devoid of any input symbols, implying a reliance on external or runtime-supplied data sources.

To define the execution logic of the `Tracker.java` module, we can break it down into the following steps:

1. **Initialization:** The module is instantiated with an empty symbol set, indicating that any data to be processed will be supplied at runtime or through external interfaces.
2. **Data Ingestion:** The module ingests data from external sources, which may include APIs, databases, file systems, or other modules. This data is presently not restricted to any particular format, given the absence of predefined symbols.

3. **Data Processing:** Upon receiving data, the module applies processing logic, which may include filtering, mapping, reducing, or other transformations. However, without predefined symbols or type information, the module's processing capabilities are dynamically determined at runtime.
4. **Data Storage or Forwarding:** After processing the data, the module either stores the results in a local or external repository or forwards the processed data to downstream modules for further processing or analysis.

The Tracker.java module's execution logic can be represented by the following high-level pseudocode:

```
public class Tracker {
    // Initialize the tracker with an empty symbol set
    public Tracker() {}

    // Ingest data from external sources
    public void ingestData(Object data) {
        // Apply processing logic to the ingested data
        Object processedData = process(data);

        // Store or forward the processed data
        storeOrForward(processedData);
    }

    // Dynamically determined processing logic based on the supplied data
    private Object process(Object data) {
        // Implement filtering, mapping, reducing, or other transformations
        // The actual logic is determined at runtime based on the data and any
        external configurations
    }

    // Store the processed data in a repository or forward it to downstream
    modules
    private void storeOrForward(Object data) {
        // Implement data storage or forwarding logic
    }
}
```

The actual implementation details of the `Tracker.java` module will depend on the specific requirements and constraints of the project, including the data formats, processing logic, and storage or forwarding mechanisms. However, the above pseudocode provides a general outline of the module's execution logic.

3.6 TrackerIntf.java

The TrackerIntf.java module presently executes with a primary focus on processing and handling data.

Upon initialization, TrackerIntf.java does not contain any defined symbols, indicating that its execution logic centers around external data ingestion. We define the execution logic as follows:

1. **Data Ingestion:** The module receives data from an external source, which may be in the form of API calls, file inputs, or other data streams.

2. **Data Validation:** TrackerIntf.java validates the ingested data to ensure it conforms to predefined formats and standards. This validation step is crucial for preventing data corruption or inconsistencies.
3. **Data Processing:** Upon successful validation, the module processes the data according to predetermined rules and algorithms. However, as no symbols are presently defined in TrackerIntf.java, the specifics of this processing step remain implementation-dependent.
4. **Data Transformation:** As part of the processing step, TrackerIntf.java transforms the data into a usable format, which may involve data type conversions, aggregation, or filtering.
5. **Data Storage or Transmission:** The final step involves storing the processed data in a designated repository or transmitting it to downstream systems for further analysis or processing.

In its current form, TrackerIntf.java serves as a data processing interface, providing a foundation for future implementation of specific processing rules and algorithms.

3.7 start_tracker.sh

The `start_tracker.sh` script serves as a bootstrapping mechanism to initiate the tracking component of the project infrastructure. This shell script is responsible for executing a series of commands that configure and launch the tracker, ensuring it is properly set up and running.

Upon invocation, `start_tracker.sh` performs the following key functions:

1. **Environment setup:** It configures the environment variables required by the tracker, such as API keys, logging settings, and other dependencies.
2. **Dependency checks:** The script verifies that all necessary dependencies are installed and available, ensuring the tracker can function correctly.
3. **Tracker initialization:** `start_tracker.sh` initializes the tracking component, which involves loading the required configuration files, setting up logging mechanisms, and establishing connections to dependent services.
4. **Execution:** The script launches the tracker, allowing it to begin collecting and processing data as intended.

By automating the startup process, `start_tracker.sh` simplifies the deployment and management of the tracking component, ensuring consistency and reliability across different environments. Its role is crucial in maintaining the overall integrity and functionality of the project infrastructure.
